TTK 4192 - Mission Planning for Autonomous Systems

Experiments with Turtlebot3

Objective:	Perform autonomous mission with Turtlebot3.
Date:	14th April 2026
Place:	D0042, IE, NTNU, Trondheim, Norway.

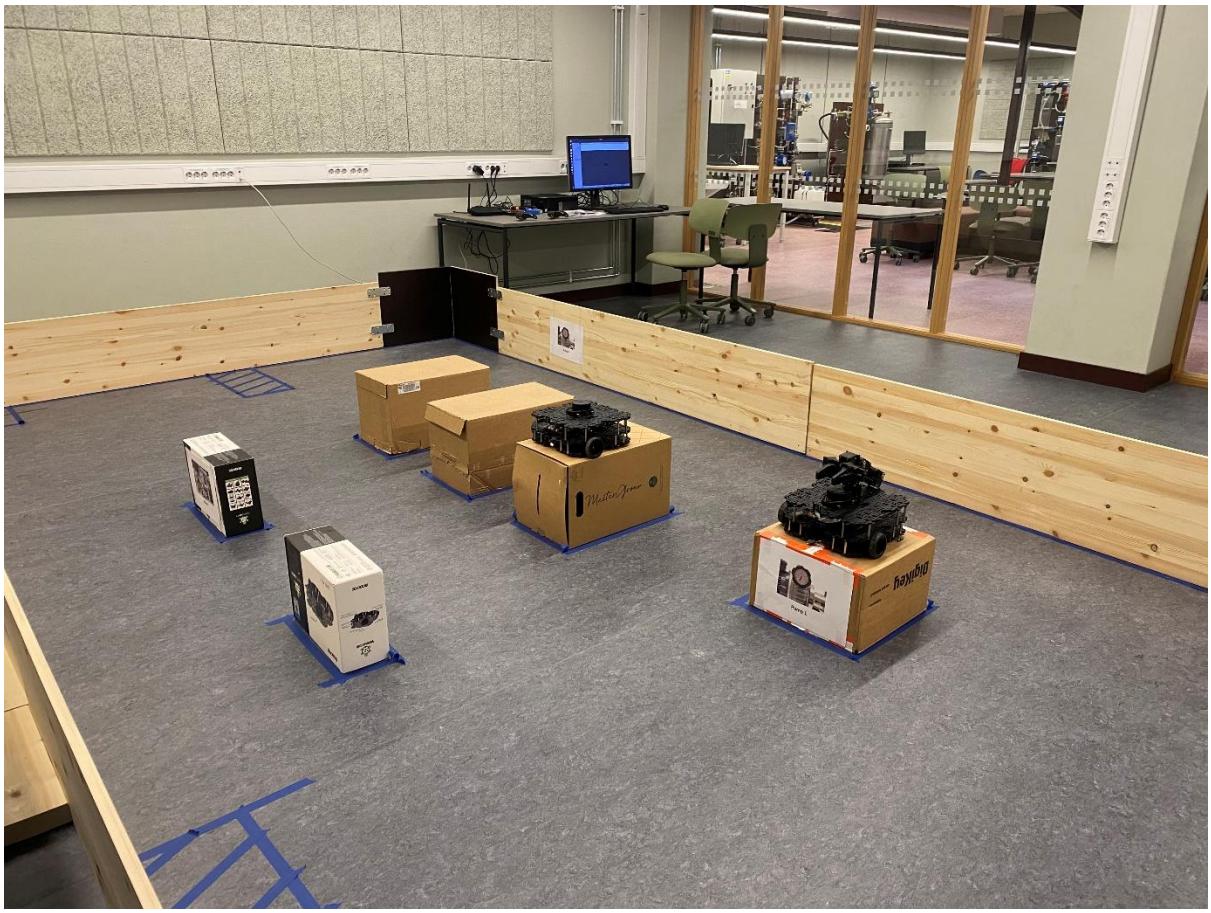


Figure: Robot and map for experiments

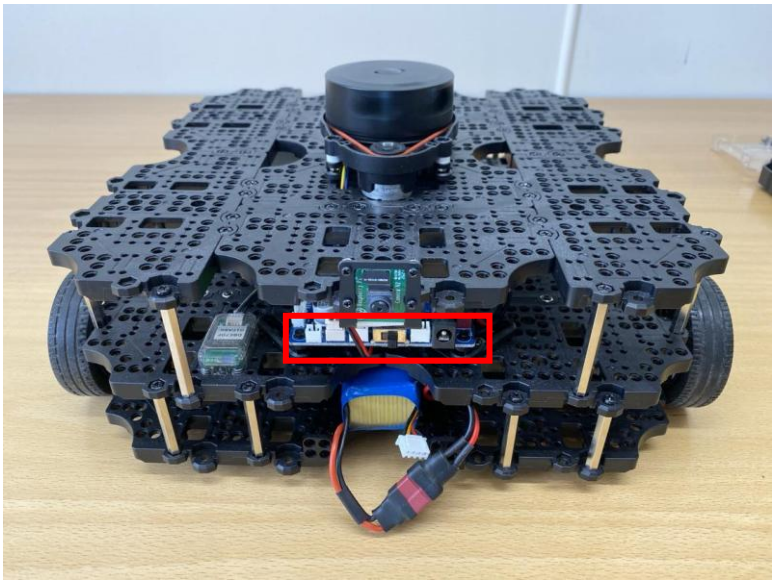
General recommendation:

Most of the commands and installation instructions are taken from ROBOTIS (Turtlebot manufacturer), so it is highly recommended to read the commands beforehand.
https://emanual.robotis.com/docs/en/platform/turtlebot3/quick-start/

PART I: Setting up computer and robot.

a) Charge the robot battery (if it is needed). To charge the robot battery use the connector in figure. The “red light” indicates charging, the “green led” means charge completed.

b) Remove battery from charger (use a tool to **avoid damage** battery), then place the battery in TurtleBot and turn on the button switch (see figure). The Lidar of the robot should start spinning.



c) Place the robot at initial position (map position [0,0])

d) Turn on the Lab computer and log in:
User: **ttk4192**
password: **ttk4192**

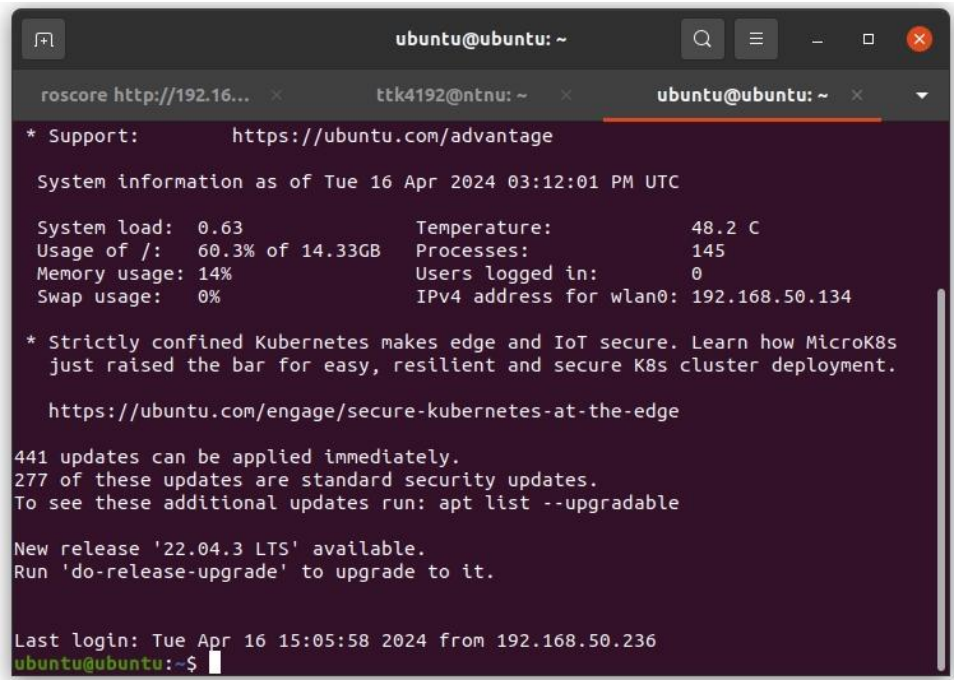
Part II: Preliminary tests with the robot

a) Choose the robot that you want to use:
Robot 1: Robot with manipulator (recommend)
IP address: 192.168.50.239

Robot 2: Robot without manipulator
IP address: 192.168.50.134

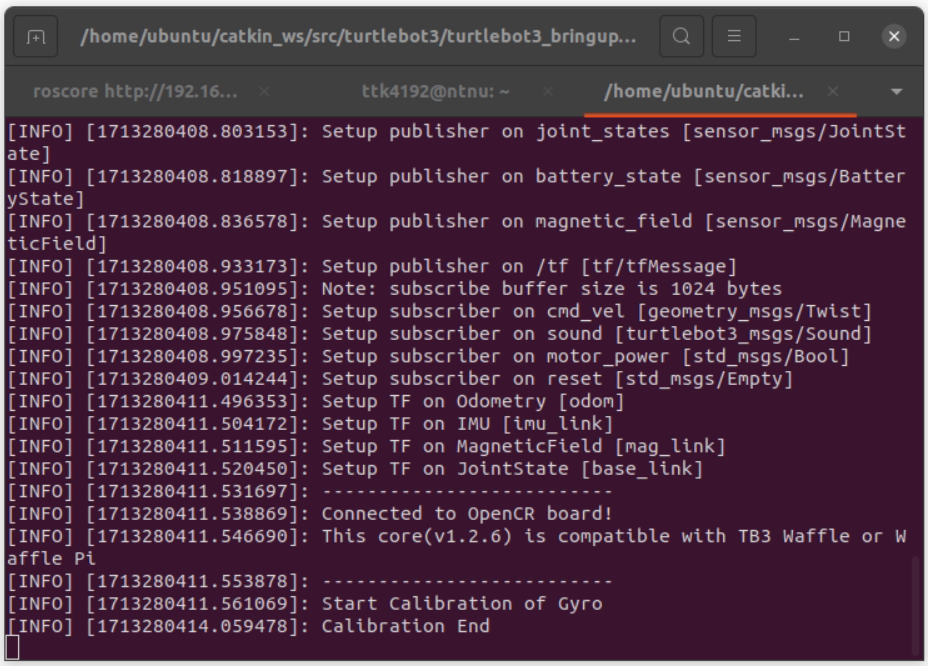
b) Open a terminal and execute the below commands (for **robot 1**):

```
$ roscore
$ ssh ubuntu@192.168.50.239
password: turtlebot
```



c) Initialize the robot, in the same windows of the previous (**physical robot**) commands execute:

```
$ export TURTLEBOT3_MODEL=waffle_pi
$ roslaunch turtlebot3_bringup turtlebot3_robot.launch
```



d) Open a new Tab and execute the “teleoperation mode” below commends (**in the Lab PC**):

```
$ export TURTLEBOT3_MODEL=waffle_pi
$ roslaunch turtlebot3_teleop turtlebot3_teleop_key.launch
```

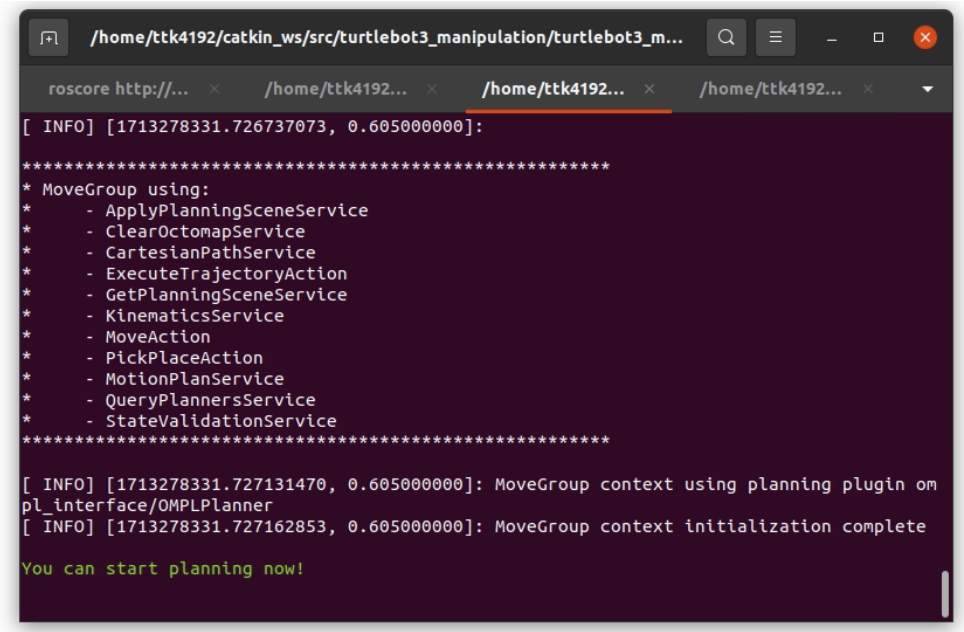
e) Now, you can move the robot with the keywords.

f) Move the robot with the joysticks.

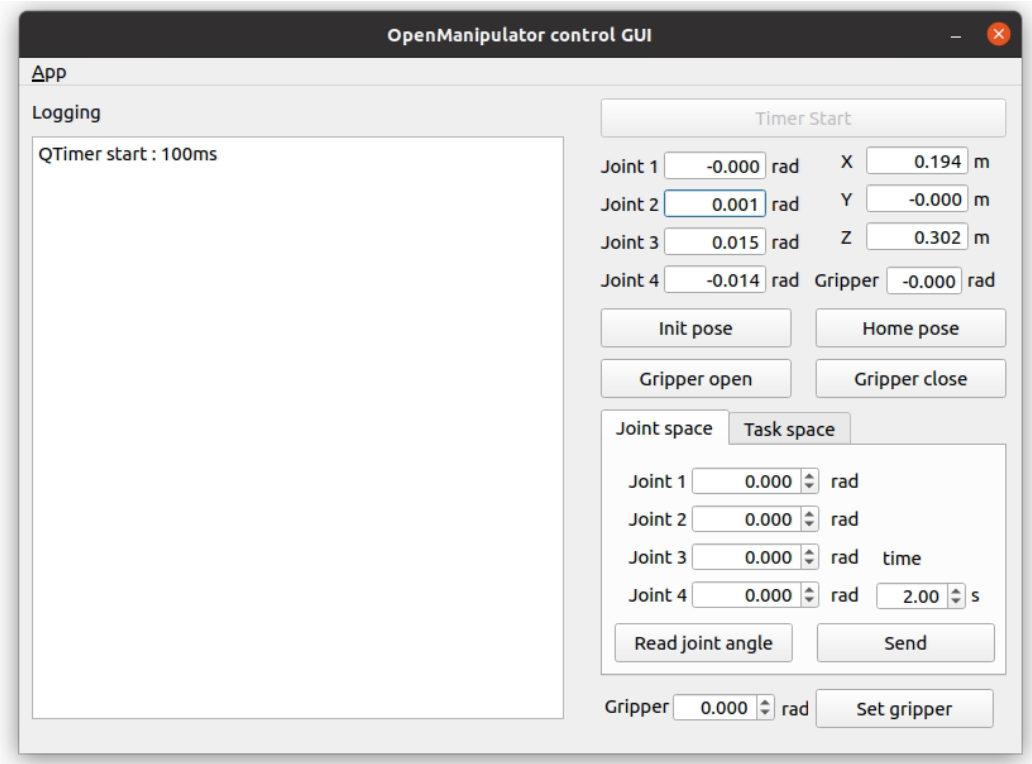
Check which joystick belong to each robot in the label back of the joystick.

g) Testing the manipulator, Stop the teleoperation tab (CTRL + C), and execute in the **LAB PC**.

```
$ roscore
$ export TURTLEBOT3_MODEL=waffle_pi
$ roslaunch turtlebot3_manipulation_moveit_config move_group.launch
roslaunch turtlebot3_manipulation_bringup turtlebot3_manipulation_bringup.launch
$ roslaunch turtlebot3_manipulation_gui turtlebot3_manipulation_gui.launch
```



h) Use the OpenManipulator control GUI to move to move the turltebot3 manipulator.



i) Moving the robot in the map, using the function "**turtlebot_move()**" provided in the assignment 4.

Open a new terminal and start the robot.

```
$ roscore
$ ssh ubuntu@192.168.50.239
password: turtlebot
$ export TURTLEBOT3_MODEL=waffle_pi
$ roslaunch turtlebot3_bringup turtlebot3_robot.launch
```

Open a new tab and execute the python script for example (which will move the robot from starting position x0= **[0,0]** to waypoint = **[1,2].**

```
$ cd ~/bkp-files/assigment4_ttk4192/scripts
$ python3 mission_planner_ttk4192.py
```

IMPORTANT:
In a new tab, execute
$ roscore
$ go to plugins>>topics>>topic monitor
$ check the /odom >> pose >> position, and verify that the robot position is at position around x=1 and y=2

☑ /odom	nav_msgs/Odometry	21.63KB/s	30.00	
child_frame_id	string			'base_footprint'
header	std_msgs/Header			
pose	geometry_msgs/PoseWithCovariance			
covariance	float64[36]			(1e-05, 0.0, 0.0, 0.0, 0.0, 0.0, 0
pose	geometry_msgs/Pose			
orientation	geometry_msgs/Quaternion			
position	geometry_msgs/Point			
x	float64			0.9728623084446182
y	float64			1.9891174856651361
z	float64			-0.0010083593706486892
twist	geometry_msgs/TwistWithCovariance			

Part III: TTK4192 computer assignment 4

Now you should try to execute **your code developed** for computer assignment 4. The procedure for running it will be the same as with the simulator. The only difference is that this time, instead of running Gazebo, you will start the physical robot.

a) Copy your packages into the "catkin_ws" directory.
   - Copy your Temporal planner's domain and problem.
   - Copy your AI_planning_TTK4192 folder (delete any other with the same name).

Open a new terminal and do catkin_make.

```
$ cd ~/catkin_ws && catkin_make
```

b) Repeat steps in part II to initialize the physical robot:

```
$ roscore
$ ssh ubuntu@192.168.50.239
password: turtlebot
```

c) Place the robot in the corner of the map (x_0 = [0,0]) in the physical map.

d) Open a new tab and run your main script.
   For example:

```
$ export TURTLEBOT3_MODEL=waffle_pi
$ rosrun Assigment4_ttk4192 assigment4_ttk4192.py
```

Q & A:

- The robot has stop to work.
  Probably is low battery, turn off and replace with the other charge battery.
- The joystick does not work.
  Check if the joystick corresponds to the robot that you are trying to control or check if the batteries are not low.
- My robot is moving in Gazebo but not in physical.
  You can only run either the physical robot or the gazebo simulator, not both simultaneously.
- The robot has touch and move the obstacles.
  Place back the obstacles and ensure that they are in the right position, see the map in the paper.
- My robot is moving very slow
  Increase the speed of the robot in the GNC code

Optional: using SLAM module

You have the option to use the SLAM module of TurtleBot for the path-finding parts, for that you will need to construct the SLAM map, with the below commands.

a) Repeat steps in part II to initialize the robot
   $ roscore
   $ ssh ubuntu@192.168.50.239
   password: turtlebot
   $ export TURTLEBOT3_MODEL=waffle_pi
   $ roslaunch turtlebot3_bringup turtlebot3_robot.launch

b) Run the SLAM module for turtlebot with manipulator

```
$ export TURTLEBOT3_MODEL=waffle_pi
$ roslaunch turtlebot3_manipulation_slam slam.launch
$ Navigate the robot at very low speed
```

c) Save the map
```
$ rosrun map_server map_saver -f ~/map_ttk4192
```

d) Open the teleoperation mode and explore the map.
e) Test your map, with Rviz

```
$ export TURTLEBOT3_MODEL=waffle_pi
$ roslaunch turtlebot3_navigation turtlebot3_navigation.launch
map_file:=$HOME/map_ttk4192.yaml
```

f) In *rviz* use the tool “2D navigation”